

Module 5 Report: Grad Cafe Security and CI

Rida Fatima Alvi - JHU Software Concepts

Install and Run with pip

From the repository root, create a virtual environment, install dependencies, and install the project in editable mode.

```
python3 -m venv module_5/.venv
source module_5/.venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r module_5/requirements.txt
cd module_5
python -m pip install -e .
```

Set database environment variables before running the app. The app supports DB_HOST, DB_PORT, DB_NAME, DB_USER, and DB_PASSWORD. DATABASE_URL is still supported for tests or CI overrides.

```
python src/app.py
# open http://127.0.0.1:8080/analysis
```

Install and Run with uv

The alternate install path uses uv so a clean environment can be recreated from requirements.txt.

```
python3 -m venv module_5/.venv
source module_5/.venv/bin/activate
python -m pip install uv
uv pip sync module_5/requirements.txt
cd module_5
uv pip install -e .
```

The uv path keeps the environment synchronized with the requirements file, which reduces local/CI dependency drift.

Dependency Graph Summary

The dependency graph is saved as module_5/dependency.svg and was generated with pydeps and Graphviz. The graph centers on src/app.py because that file creates the Flask application, defines routes, and renders the analysis page. Flask appears as a major dependency because it provides routing, request handling, JSON responses, and template rendering. psycopg appears because the application connects to PostgreSQL for database reads and writes. query_data.py appears because it computes the values shown on the analysis page. load_data.py appears because it creates the applicants table and inserts new scraped records. db_config.py appears because database credentials are read from environment variables instead of being hard-coded into the application.

SQL Injection Defenses

SQL construction was refactored in module_5/src/load_data.py and module_5/src/query_data.py. Dynamic SQL text is composed with psycopg sql.SQL objects instead of f-strings, concatenation, or .format() with raw user input. Table names and column names use sql.Identifier so database identifiers are quoted safely. User values are passed separately to cursor.execute(statement, params), so values are parameterized instead of being inserted into SQL text. This separates statement construction from execution and prevents malicious input from changing the meaning of the SQL command.

Required SQL Behaviors Met

Every query has an explicit LIMIT. User-controllable limits pass through `clamp_limit()`, which enforces a maximum of 100 and a minimum of 1. Statements are built as composed SQL objects, and execution parameters are passed separately. Inserts use named placeholders created with `sql.Placeholder` and pass a row dictionary to `cursor.execute`. Read queries use `%s` placeholders and parameter lists for values such as limits and filters.

Least-Privilege Database Configuration

The application is not read-only because the Pull Data button writes new applicant rows into PostgreSQL. The least-privilege application role is `gradcafe_app`. It is a login role but is not a superuser, cannot create databases, cannot create roles, cannot replicate data, and does not own the applicants table. The app user is granted `CONNECT` on `gradcafe_db`, `USAGE` on the public schema, and `SELECT` and `INSERT` on `public.applicants`. These permissions are enough for the analysis page to read results and for Pull Data to insert rows, while avoiding `DROP`, `ALTER`, `UPDATE`, `DELETE`, `TRUNCATE`, `CREATE`, and owner-level permissions.

```
GRANT CONNECT ON DATABASE gradcafe_db TO gradcafe_app;  
GRANT USAGE ON SCHEMA public TO gradcafe_app;  
REVOKE CREATE ON SCHEMA public FROM gradcafe_app;  
GRANT SELECT, INSERT ON TABLE public.applicants TO gradcafe_app;
```

Security and CI Evidence

Pylint evidence is saved as `module_5/pylint_score.txt` and shows a 10.00/10 score. The dependency graph is saved as `module_5/dependency.svg`. Snky dependency scan evidence is saved as `module_5/snyk-analysis.png`. Snky Code evidence is saved as `module_5/snyk-code-results.png`. The GitHub Actions workflow is saved as `.github/workflows/ci.yml` and includes separate jobs for Pylint, Pytest with PostgreSQL, dependency graph generation, and Snky output.

Deliverable Checklist

Deliverable	File
10/10 lint evidence	<code>module_5/pylint_score.txt</code>
Dependency graph	<code>module_5/dependency.svg</code>
Snyk dependency evidence	<code>module_5/snyk-analysis.png</code>
Installable package config	<code>module_5/setup.py</code>
Environment placeholder file	<code>module_5/.env.example</code>
CI workflow	<code>.github/workflows/ci.yml</code>